

CONTENIDO DE LA LECCIÓN 1

EL ALGORITMO DEL PROGRAMADOR, ABSTRACCIÓN Y REFINAMIENTO SUCESIVO

1. Introducción	2
2. El algoritmo del programador	2
2.1. Definición del problema	4
2.2. Planeación de la solución del problema	5
2.3. Codificación del problema	6
2.4. Verificación y depuración del programa	6
2.4.1. Prueba de escritorio del programa	7
2.4.2. Compilación y vinculación del programa	7
2.4.3. Ejecución del programa	8
2.4.4. Cómo usar un depurador (<i>debugger</i>)	8
2.5. Documentación del programa	9
2.6. Resumen del algoritmo del programado	9
3. Examen breve 1-1	10
4. Solución de problemas utilizando algoritmos	10
4.1. Ejemplos 1.1 hasta el 1.18	13
5. Examen breve 1-2	21
6. Abstracción de problemas y refinamiento sucesivo	21
7. Examen breve 1-3	24
8. Solución de problemas en acción: <i>Teorema de Pitágoras</i>	24
8.1. Problema	24
8.2. Definición del problema	24
8.3. Planeación de la solución	25
9. Solución de problemas en acción: <i>Impuesto de las ventas</i>	27
9.1. Problema	27
9.2. Definición del problema	27
9.3. Planeación de la solución	27
10. Solución de problemas en acción: <i>Interés de una tarjeta de crédito</i>	28
10.1. Problema	28
10.2. Definición del problema	28
10.3. Planeación de la solución	29
11. Lo que necesita saber	30
12. Preguntas y problemas	31
12.1. Preguntas	31
12.2. Problemas	31

LECCIÓN 1

EL ALGORITMO DEL PROGRAMADOR, ABSTRACCIÓN Y REFINAMIENTO SUCESIVO

INTRODUCCIÓN

La **programación**, en cierta forma, es la ciencia y el arte de solucionar problemas. Para ser un buen programador, debe ser bueno solucionando problemas. Para lograrlo, debe enfrentarlos en forma metódica: desde la **definición inicial** e **inspección del problema** hasta la **solución final**, **verificación** y **comentarios**. Cuando se inicia en la **programación** y se enfrenta a un problema, se verá tentado a **codificar** tan pronto como tenga una idea de cómo resolverlo. Sin embargo, **debe** resistirse a esta tentación. Tal enfoque puede funcionar para problemas simples, pero **no** ocurre lo mismo con problemas complejos.

Objetivos de esta lección:

- **Aprenderá** un método sistemático que lo convertirá en un buen solucionador de problemas y, por lo tanto, en un buen programador. A este método le llamaremos el **algoritmo** del programador.
- **Aprenderá** y **utilizará** los pasos que se requieren para resolver casi cualquier problema de programación usando el método estructurado **descendente** o de **arriba-abajo (top-down)**
- **Estudiará** el concepto de **abstracción** que se requiere para un lenguaje de computadora, que permite ver los problemas en términos generales sin la angustia de los detalles de implantación.
- A partir de una solución inicial **abstracta**, **refinará** la **solución** paso a paso hasta que alcance un nivel que pueda ser codificado directamente en un lenguaje de programación.

Asegúrese de entender este material y resuelva los problemas propuestos al final de esta lección. Conforme tenga más experiencia, encontrará que el **secreto** para programar con éxito es una buena planeación a través del **análisis abstracto** y el **refinamiento sucesivo**, lo cual dará como resultado diseños de **sistemas estructurados descendentes** o de **arriba-abajo**. Estos diseños son soportados por **C++**.

EL ALGORITMO DEL PROGRAMADOR

Antes de estudiar el **algoritmo del programador**, será útil definir lo que se entiende por algoritmo:

Un **algoritmo** es una secuencia ordenada de pasos, sin ambigüedades, que conducen a la solución de un problema dado y expresado en lenguaje natural, por ejemplo el castellano.

Todo **algoritmo** debe ser:

- **Preciso**. Indicando el orden de realización de cada uno de los pasos.
- **Definido**. Si se sigue el **algoritmo** varias veces proporcionándole los mismos datos, se deben obtener siempre los mismos resultados.
- **Finito**. Al seguir el **algoritmo**, éste debe terminar en algún momento, es decir, debe terminar en un número finito de pasos.

Los **algoritmos** no son exclusivos de la computación. Cualquier conjunto de instrucciones, como las que se encuentran en una receta de cocina o guía de ensamblaje, pueden ser considerados **algoritmos**. El **algoritmo del programador** es una receta que el programador sigue cuando desarrolla programas.

Debemos aclarar, sin embargo, que diseñar la solución de problemas computacionales suele ser una tarea difícil. No existe un conjunto completo de reglas, ni algún **algoritmo** que nos diga como solucionar problemas. El **diseño** de problemas es un proceso creativo; no obstante, existe un bosquejo del plan a seguir. La **figura 1.1** presenta en forma esquemática dicho bosquejo.

Fase de resolución de problemas

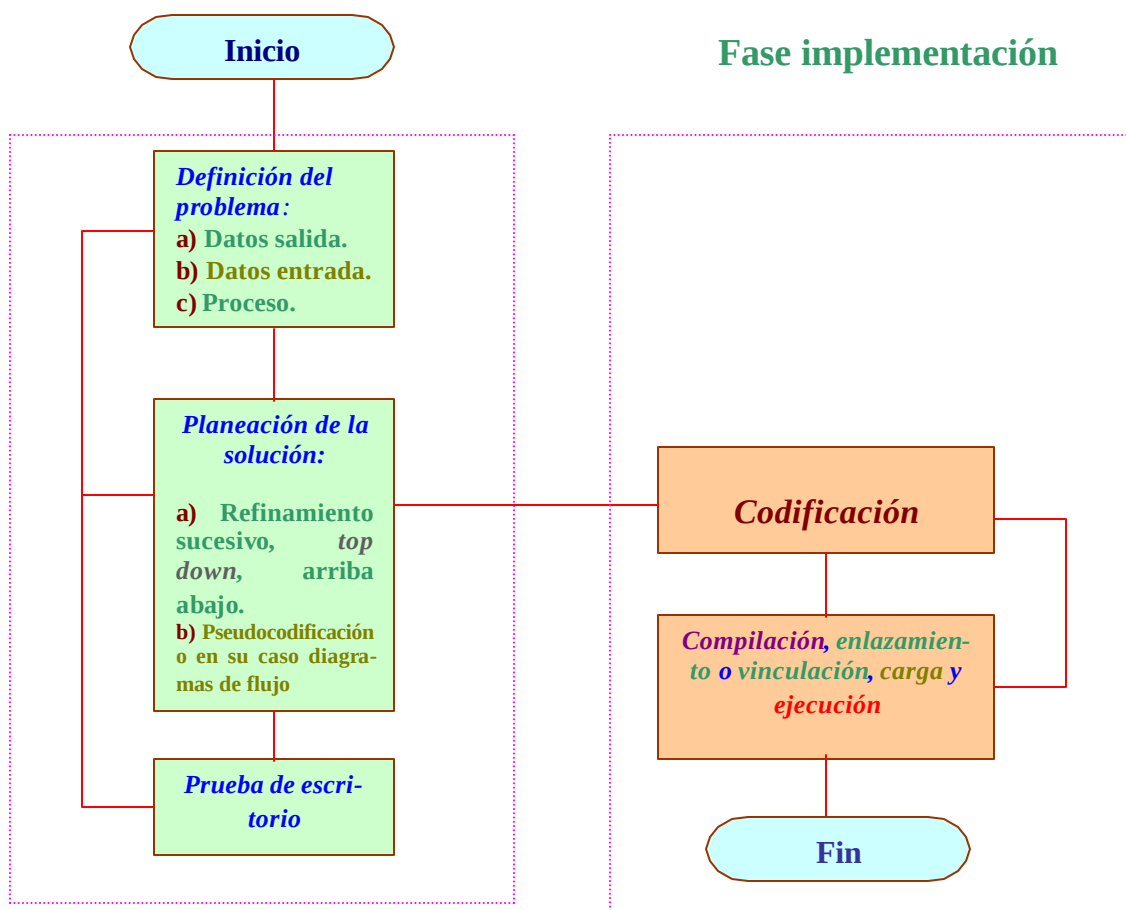


Figura 1.1. Diagrama del **Algoritmo** del Programador

Como se indica en la **figura 1.1**, todo el proceso de diseño de programas (el **algoritmo del programador**) se puede dividir en dos fases: La **fase de resolución de problemas** y la **fase de implementación**. El resultado de la fase de resolución de problemas es un algoritmo expresado en español, para resolver el problema. Para obtener un programa escrito en un lenguaje de programación como **C++**, el algoritmo debe traducirse al lenguaje de pro-

gramación. La producción del programa final, a partir del algoritmo, es la *fase de implementación*.

En forma lineal podemos decir que el *algoritmo del programador* consta básicamente de **cinco** pasos:

- *Definición del problema.*
- *Planeación de la solución del problema.*
- *Codificación del programa.*
- *Verificación y depuración del programa.*
- *Documentación del programa.*

DEFINICIÓN DEL PROBLEMA

El primer paso es cerciorarse de que la tarea, es decir, lo que queremos que el programa haga, esté especificado de forma completa y precisa. No tome este paso a la ligera. Si no sabe exactamente que quiere obtener como salida de su programa, podría sorprenderse lo que el programa produce. Debemos asegurarnos que sabemos que entradas tendrá exactamente el programa y que información se supone que debe estar en la salida, así como la forma que debe tener dicha información.

La *definición del problema* es un paso obvio en la solución de cualquier problema. Sin embargo, a menudo es el que más se pasa por alto, en especial en programación. La deficiencia de una buena definición del problema a menudo tiene como resultado *tejer en el aire*, especialmente en aplicaciones complejas de programación.

Piense en un problema típico de programación: **controlar el inventario de una gran tienda**. ¿Qué debe considerar como parte de la definición del problema?

➤ Probablemente la primera *consideración* es *¿qué se quiere lograr del sistema?*

- ♦ La información consiste en un reporte impreso del inventario o adicionalmente, *¿el sistema generará automáticamente las órdenes del producto basado en las ventas? ¿Debe guardarse en disco cualquier información generada por la transacción de un cliente o puede ser descartada? ¿Qué tipo de datos se encuentran en la información de salida o en que consiste? ¿Es un dato numérico, de tipo carácter o ambos? ¿Qué formato deben tener los datos de salida?* Todas estas preguntas deben contestarse para *definir los requerimientos de salida*.

➤ La segunda *consideración* es: *dado los requerimientos de salida, ¿Cuáles son los requerimientos de entrada?*

- ♦ Los requerimientos de salida por lo general sugieren lo que debe de *introducirse* (*requerimientos de entrada*) dentro del sistema. Por ejemplo, en nuestro problema de inventario, una salida podría ser un resumen de las compras de los clientes. *¿Cómo se ingresan estas compras en el sistema? ¿El dato se obtiene desde el teclado o es producto de la información que se ingresa en forma automática por un sistema óptico de reconocimientos de caracteres (OCR) que lee el precio del producto a través de código de barras?* La entra-

da consiste en ¿datos numéricos, datos de tipo carácter o una combinación de ambos? ¿Cuál es el formato de los datos?

➤ La tercera **consideración** es: dados los requerimientos de salida y los de entrada ¿Cuáles son los procesos que deben de realizarse con las entradas para obtener las salidas deseadas?

- ♦ Para nuestro ejemplo de inventario, la mayor parte del procesamiento de los clientes ¿Se hace en la terminal de la caja registradora o será manejada por una computadora central de la tienda? ¿Qué hay con respecto a la verificación de la tarjeta de crédito y los registros de inventarios? ¿Se hará este proceso en una microcomputadora local, una minicomputadora localizada dentro de la tienda o una macrocomputadora localizada en otra parte de la ciudad? ¿Qué clase de programas se escribirán para realizar el procesamiento y quién los escribirá? ¿Qué clase de cálculos y decisiones deben hacerse a los datos dentro de los programas individuales para llevar a cabo la salida deseada?

Todas estas preguntas deben contestarse cuando se define cualquier problema de programación. En resumen: la **definición del problema** debe considerar los requerimientos de **salida**, **entrada** y **procesamiento**. El problema de inventario claramente requiere una definición precisa. Sin embargo, incluso en aplicaciones pequeñas, debe todavía considerarse el tipo de **salida**, **entrada** y **procesamiento** que requiere el problema.

Cuando se define un problema, se buscan los **nombres** y **verbos** dentro de la declaración del mismo. Los **nombres** a menudo sugieren información de **entrada** y **salida**, los **verbos** sugieren pasos de **procesamiento**. En cualquier caso, la aplicación siempre sugerirá la **definición del problema**.

PLANEACIÓN DE LA SOLUCIÓN DEL PROBLEMA

La etapa de **planeación** asociada con cualquier problema es tal vez la más importante de la solución. Imagínese construyendo una casa sin un buen número de planos ¡Los resultados podrían ser catastróficos! Lo mismo sucede al desarrollar sistemas de información sin un buen plan. Cuando se desarrollan sistemas de información, la etapa de planeación se implementa usando una serie de algoritmos. Cuando se planean programas de computadora, se utilizan los algoritmos para esquematizar los pasos de la solución usando declaraciones en **lenguaje natural** como el castellano, llamadas **seudo código**, que requieren menos precisión que un **lenguaje formal** de programación. Un buen algoritmo en pseudo código debe ser independiente pero fácilmente traducible a cualquier lenguaje formal de programación. El prefijo pseudo se usa para resaltar que no se pretende que este código sea compilado y ejecutado en una computadora. La razón para usar pseudo código es que nos permite transmitir en términos generales las ideas básicas de un algoritmo. Una vez que los programadores entienden el algoritmo que está siendo expresado por el pseudo código, pueden implementarlo en el lenguaje de programación de su elección. Esta es, en esencia, la diferencia entre el pseudo código y un programa de computadora. Un programa en pseudo código simplemente expone los pasos necesarios para realizar algún cómputo mientras que el programa informático correspondiente es la traducción de estos pasos en la **sintaxis** de un lenguaje de programación particular.

Encontraremos el pseudo código útil cuando queramos expresar un algoritmo sin preocuparnos acerca de cómo el algoritmo será implementado. Esta posibilidad de ignorar los detalles de implementación usando pseudo código facilitará el análisis al permitir que nos concentremos únicamente en los aspectos computacionales de un algoritmo.

El pseudo código básicamente consiste en **palabras reservadas** y frases afines al castellano que se utilizan para indicar el flujo de control.

CODIFICACIÓN DEL PROGRAMA

La **codificación** del programa es una de las actividades más sencillas dentro del proceso de programación, siempre y cuando se haya hecho un buen trabajo en la definición del problema y la planeación de la solución. La codificación implica la escritura real del programa en un lenguaje formal de programación. El lenguaje que se utilice será determinado por la naturaleza del problema, los lenguajes disponibles y los límites de su sistema de cómputo. Una vez que se selecciona un lenguaje, se escribe o codifica el programa, traduciendo los pasos de su algoritmo en código de lenguaje formal.

Sin embargo, la **codificación** es en realidad un proceso mecánico y debe ser considerada secundaria para desarrollar algoritmos. En el futuro, las computadoras generarán sus propios códigos de programa a partir de algoritmos bien contruidos. La investigación en el campo de la **inteligencia artificial** ha dado origen al **software** de **generación de código**. Es necesario recordar que si bien las computadoras algún día puedan generar sus propios códigos de programación a partir de algoritmos, serán indispensables la creatividad y el sentido común de un humano en la planeación de la solución y el desarrollo del algoritmo.

VERIFICACIÓN Y DEPURACIÓN DEL PROGRAMA

Pronto descubrirá que es motivo de alegría cuando un programa **corre** sin ningún error por primera vez. Por supuesto, una buena definición del problema y una buena planeación evitarán muchos errores en el programa. Sin embargo, siempre hay unas cuantas **fallas** que no son detectadas, sin importar que tan minuciosa haya sido la planeación. Quitar las fallas del programa (**depuración**) a menudo es la parte del trabajo que consume más tiempo en todo el proceso de programación. Las estadísticas muestran que a menudo más de **50%** del tiempo de un programador se consume en la depuración del programa.

No hay en absoluto un procedimiento correcto para **depurar un programa**, pero un enfoque sistemático puede ayudar a hacer más fácil ese proceso. Los pasos básicos de depuración son: **darse cuenta que tiene un error**, **localizar** y **determinar la causa del error** y **corregir el error**:

➤ **Darse cuenta que tiene un error**

- ◆ *Antes que nada, tiene que darse cuenta que tiene un **error**. Algunas veces, esto es obvio cuando su computadora se **congela** o **falla**. En otras ocasiones el programa parece trabajar bien hasta que cierta información inesperada es introducida por alguien que usa el programa. El más sutil de los **errores** ocurre cuando el programa está ejecutándose bien y los resultados parecen correctos,*

pero cuando se examinan los resultados con más detalle, éstos no son correctos.

➤ Localizar y determinar la causa del error

- ♦ *El siguiente paso en el proceso de depuración es **localizar** y **determinar** la causa de los errores – algunas veces ésta es la parte más difícil. Aquí es donde entra en juego una buena herramienta de programación llamada depurador.*

➤ Corregir el error

- ♦ ***Corregir** el error es el paso final en la depuración. Su conocimiento del lenguaje **C++**, estas lecciones, la ayuda en línea de **C++**, un depurador **C++** y sus manuales de referencia **C++** son todas herramientas valiosas en la corrección del error y la eliminación de la falla en su programa.*

Cuando se programa en **C++**, hay cuatro cosas que pueden hacerse para verificar y depurar su programa: la **prueba de escritorio**, **compilación** / **vinculación**, la **ejecución** y la **depuración** del programa.

PRUEBA DE ESCRITORIO DEL PROGRAMA

La **prueba de escritorio** de un programa es similar a revisar una carta o manuscrito. La idea es seguir el programa mentalmente para asegurarse que éste trabaja en forma lógica. Debe considerar varias posibilidades de entrada y escribir cualquier resultado generado durante la ejecución del programa. En particular, trate de determinar qué hará el programa con datos no muy comunes considerando posibilidades de entrada que **no deberían** pasar. Siempre tenga presente la ley de **Murphy** cuando haga una prueba de escritorio de un programa: **si una condición dada no puede o no debe ocurrir, ¡ocurrirá!**

Por ejemplo, suponga que un programa requiere que el usuario ingrese un valor para obtener su raíz cuadrada. De seguro, el usuario **no debería** introducir un valor negativo, porque la raíz cuadrada de un número negativo, es un número complejo. Sin embargo, ¿Qué hará el programa si el usuario lo hace? Otra posibilidad que siempre debe tomarse en cuenta es el ingreso del número cero, especialmente cuando se usa como parte de una operación aritmética de división.

Cuando se inicia como programador, estará tentado a omitir la fase de prueba de escritorio, **¡Quiere correr el programa una vez que lo ha escrito!** Sin embargo, conforme adquiera más experiencia, se dará cuenta de que puede ahorrar tiempo si realiza la prueba de escritorio.

COMPILACIÓN Y VINCULACIÓN DEL PROGRAMA

En este punto, está listo para introducir el programa al sistema de cómputo. Una vez dentro, el programa debe ser **compilado** o **traducido al código de máquina**. Afortunadamente, el compilador está diseñado para verificar ciertos errores del programa. Estos por lo general, son errores de **sintaxis** que se cometen cuando se codifica el programa. Un **error de sintaxis** es una violación a las reglas del lenguaje de programación, como cuando se

usa un punto en lugar de una coma. También habrá **errores de escritura**. Un **error de escritura** ocurre cuando intenta mezclar tipos de datos diferentes, como números y caracteres. Es como tratar de sumar peras y manzanas.

Durante el proceso de compilación, se generan mensajes de advertencia y error así como la posición en donde se detectó el error en el programa. Una vez que se corrige se debe compilar el programa otra vez. Si se detectan otros errores, deberá corregirlos, volver a compilar y así sucesivamente, hasta que todo el programa no envíe mensajes de error durante la compilación.

Después, debe **vincularse o ligarse** (*link*) el programa a otras rutinas que pueden ser necesarias para su ejecución. Errores en la vinculación ocurrirán cuando tales rutinas no estén disponibles o no se puedan localizar en el directorio designado para el sistema.

EJECUCIÓN DEL PROGRAMA

Una vez que el programa se ha compilado y vinculado, deberá **ejecutarlo o correrlo**. Sin embargo, sólo porque el programa se ha compilado y vinculado no significa que correrá con éxito dentro de todas las posibles condiciones. Algunas fallas comunes que ocurren en esta etapa incluyen **errores lógicos** y en **tiempo de ejecución**. Estos son los más difíciles de detectar. Un **error lógico** ocurrirá, por ejemplo, cuando un ciclo le dice a la computadora que repita una operación, pero no le dice cuándo deje de repetirla. Esto se denomina **ciclo infinito**. Esta falla no generará mensaje de error, porque la computadora simplemente hace lo que se le dijo que hiciera. La ejecución del programa debe detenerse y depurarse antes de ejecutarlo nuevamente.

Un **error en tiempo de ejecución** ocurre cuando el programa intenta realizar una operación ilegal, como las definidas por las leyes de las matemáticas o del compilador en uso. Dos errores en tiempo de ejecución comunes en matemáticas son la división por cero y el intento de obtener la raíz cuadrada de un número negativo. Un error común impuesto por el compilador es el intento de manejo de un valor entero fuera de rango. La mayoría de los compiladores **C++** limitan a los enteros a un rango de **-32768** a **+32767**. Pueden ocurrir resultados impredecibles si un valor entero excede este rango.

Algunas veces cuando se presenta un error en tiempo de ejecución el programa aborta en forma automática mostrando un mensaje de error. Otras veces, el programa parece ejecutarse en forma apropiada, pero genera resultados incorrectos, comúnmente llamados **basura**. Otras veces deberá consultar el manual de referencia del compilador para determinar la naturaleza exacta del problema. El error debe ser localizado y corregido antes de hacer otro intento para ejecutar el programa.

COMO USAR UN DEPURADOR (DEBUGGER)

Una de las herramientas más importantes de programación es el **depurador**. Un **depurador** proporciona una vista microscópica de lo que sucede en su programa. La mayoría de los compiladores **C++** incluyen uno preconstruido o integrado, que le permiten declaraciones de programa de un solo paso y ver los resultados de la ejecución en la **memoria** y la

CPU (por sus siglas en inglés-**U**nidad **C**entral de **P**rocesamiento) Queda fuera de los límites de estos apuntes el tratar con detalle el uso de un depurador.

DOCUMENTACIÓN DEL PROGRAMA

El paso final en el algoritmo del programador a menudo se pasa por alto, pero probablemente es uno de los pasos más importantes, en especial en la programación comercial. La **documentación** es fácil si se ha hecho un buen trabajo en la **definición del problema**, la **planeación de la solución**, la **codificación**, la **verificación** y la **depuración del programa**. La documentación final del programa es simplemente el registro de estos pasos de programación. Una buena documentación deberá incluir como mínimo lo siguiente:

- Una **descripción de la definición del problema** que incluya el tipo de **entrada**, de **salida** y el **procesamiento** que emplea el programa.
- Los **algoritmos** utilizados en la solución del problema.
- Un **listado** del programa que incluya un esquema con **comentarios** claros. Los comentarios dentro del programa son una parte importante del proceso de documentación. Cada programa deberá incluir comentarios al principio para explicar lo que hace, cualquier algoritmo especial que se emplee y un resumen de la definición del problema. Además, deberá incluirse el nombre del programador, la fecha en que se escribió el programa y la última modificación.
- **Muestras** de datos de entrada y salida.
- **Resultados** de la verificación y la depuración.
- **Instrucciones** para el usuario.

La **documentación** debe ser clara y estar bien organizada. Debe entenderse con facilidad tanto por usted como por cualquier otra persona que tenga necesidad de usar o modificar su programa. Qué tan bueno será un ingenioso programa si nadie puede determinar ¿qué hace, cómo se usa o cómo darle mantenimiento?

Los comentarios siempre deberán ser un proceso continuo. En cualquier momento que trabaje o modifique un programa, cerciórese de que los comentarios estén actualizados para reflejar sus experiencias y modificaciones.

RESUMEN DEL ALGORITMO DEL PROGRAMADOR

Muchos programadores novatos no entienden la necesidad de diseñar un algoritmo antes de escribir un programa en un lenguaje de programación como **C++**, y tratan de abreviar el proceso omitiendo por completo la fase de resolución de problemas, o reduciéndola simplemente a la parte de **definición del problema**. Esto parece razonable, ¿Por qué no *ir por todo* y ahorrar tiempo? La respuesta es que *¡no se ahorra tiempo!* La experiencia ha demostrado que el proceso de dos fases produce un programa que funciona correctamente en menos tiempo. El proceso de dos fases simplifica la fase de diseño del algoritmo aislándola de las reglas detalladas de un lenguaje de programación como **C++**. El resultado es que el proceso de diseño de algoritmos se vuelve mucho menos intrincado y mucho menos propenso a errores. Aunque el programa no sea muy grande, seguir debidamente los pasos puede representar la diferencia entre medio día de trabajo cuidadoso y varios días frustrantes de búsqueda de errores en un programa que no se entiende bien.

La fase de implementación no es un paso trivial. Hay detalles de los que debemos ocuparnos y a veces tales detalles son sutiles, pero es mucho más sencillo de lo que podría pensar al principio. Una vez que se familiarice con **C++** o cualquier otro lenguaje de programación, la traducción de un algoritmo del español al lenguaje de programación se volverá una tarea rutinaria.

En ambas fases se efectúan pruebas. Antes de escribir el programa, el algoritmo se prueba y, si resulta deficiente, se vuelve a diseñar. Estas pruebas de escritorio se efectúan siguiendo mentalmente el algoritmo y ejecutando los pasos nosotros mismos. Si el algoritmo es grande necesitaremos lápiz y papel. El programa en **C++** se prueba compilándolo y ejecutándolo con datos de muestra. El compilador genera mensajes de error cuando detecta ciertos tipos de errores. Para encontrar otros tipos de errores, es preciso verificar que las salidas sean correctas.

El proceso de la **figura 1.1** es una representación idealizada del proceso de diseño de programas. Es la imagen básica lo que debemos tener en mente, pero la realidad a veces es más complicada. En la realidad, se descubren equivocaciones y deficiencias en puntos inesperados, y podría ser necesario regresar y rehacer un paso previo. Por ejemplo, las pruebas del algoritmo podrían revelar que la definición del problema fue incompleta. En tal caso hay que retroceder y reformular la definición. A veces no se observan deficiencias en la definición del algoritmo hasta que se prueba el programa. En tal caso es necesario retroceder y modificar la definición o el algoritmo y todo lo que les sigue en el proceso de diseño.

EXAMEN BREVE 1-1

SOLUCIÓN DE PROBLEMAS UTILIZANDO ALGORITMOS

En la sección anterior aprendió que un algoritmo es una secuencia de instrucciones paso a paso, que producirá la solución a un problema.

Por ejemplo, considere la siguiente serie de instrucciones:

- **Humedezca** su cabellera.
- **Aplique** con la espuma un suave masaje a los cabellos.
- **Enjuague** evitando que la espuma entre en sus ojos.
- **Repita**.

*¿Le parece familiar? Por supuesto, ésta es una serie de instrucciones que se encuentran en la etiqueta de una botella de champú. Pero, ¿se ajusta a la definición técnica de algoritmo? En otras palabras, ¿produce un resultado? Es posible que diga **si**, pero vea más de cerca. El algoritmo requiere que usted se mantenga repitiendo el procedimiento un número infinito de veces, así que, teóricamente ¡nunca deberá parar de lavar su cabello! Un buen algoritmo de computadora **debe terminar en un tiempo finito**. La instrucción de repetición puede alterarse con facilidad para hacer el algoritmo del champú técnicamente correcto:*

Repita hasta que el pelo esté limpio

Para terminar el proceso de lavado con champú deberá decidir cuándo su cabello está limpio.

La analogía anterior del champú parece algo trivial. Es probable que piense que cualquier persona inteligente no repetirá el proceso de lavado con champú un número infinito de veces, ¿no es cierto? Es obvio en el caso de que un humano ejecute el algoritmo, porque tenemos sentido común. Pero ¿qué pasa con una computadora? Las computadoras hacen exactamente lo que se pide que haga en el programa. Como resultado, una computadora deberá repetir el algoritmo original de lavado con champú una y otra vez un número infinito de veces. Esta es la razón por la que los algoritmos que se escriben para programas deben de ser precisos.

Ahora vamos a desarrollar un algoritmo para el **envío de una carta por correo**. Piense en los pasos que incluye este sencillo proceso. **Primero debe escribir la dirección en el sobre, doblar la carta, meter la carta en el sobre y cerrar el sobre con pegamento. Necesita también una estampilla, si no tiene una, tendrá que comprarla. Una vez que tenga la estampilla, debe colocarla en el sobre y enviar la carta.** El siguiente algoritmo resume los pasos de este proceso:

- **Obtener** un sobre.
- **Escribir** la dirección en el sobre.
- **Doblar** la carta.
- **Meter** la carta en el sobre.
- **Cerrar** el sobre.
- En caso de no tener una estampilla, entonces **comprar** una.
- **Pegar** la estampilla en el sobre.
- **Enviar** la carta por correo.

¿Esta secuencia de instrucciones se ajusta a nuestra definición de un buen algoritmo? En otras palabras, ¿produce la secuencia de instrucciones un resultado en una cantidad finita de tiempo? Sí, suponiendo que cada operación pueda ser entendida y llevada a cabo por la persona que envíe la carta por correo. Esto trae dos características adicionales de los buenos algoritmos: cada operación dentro del algoritmo debe ser **bien definida** y **efectiva**. Por bien definida se entiende que cada uno de los pasos debe ser claramente entendible por personas de procesamiento de datos. Por efectiva, se entiende que debe existir algún medio para llevar a cabo la operación. En otras palabras, la persona que envíe la carta debe poder realizar cada uno de los pasos del algoritmo. En el caso de un algoritmo para programa de computadora, el compilador deberá tener el medio para ejecutar cada operación del algoritmo.

En resumen, un buen algoritmo de computadora debe tener los tres atributos siguientes:

1. Empleo de **instrucciones bien definidas** que entiendan el personal de procesamiento de datos.
2. El empleo de **instrucciones que puedan llevarse a cabo** en forma efectiva por el compilador que ejecuta el algoritmo.
3. **Producir una solución** al problema en una cantidad finita de tiempo.

Para escribir algoritmos para programas, es necesario establecer una serie de operaciones **efectivas** y bien **definidas**. La serie de operaciones de pseudo código incluidas en la **tabla 1.1** conformarán nuestro lenguaje algorítmico. Usaremos estas operaciones de ahora en adelante, en cualquier momento que escribamos algoritmos para computadora.

Tabla 1.1. Operaciones de pseudo código que se utilizarán en estas lecciones

SECUENCIA	DECISIÓN	ITERACIÓN
Sumar (+)	si expresión condicional entonces	mientras expresión condicional hacer
Restar (-)	<acciones>.	<acciones>.
Incrementar		
Decrementar	si expresión condicional entonces	hacer
Multiplicar (*)	<acciones>.	<acciones>.
Dividir (/)	sino	mientras expresión condicional
Calcular	<acciones>.	
Escribir (lista_de_expresiones)		para instrucción de asignación hasta valor final
Leer (lista_de_variables)	según sea <expresion_ordinal> hacer	hacer en paso de incremento
Imprimir (lista_de_expresiones)	<lista_de_valores_ordinales>: <acciones>	<acciones>.
Asignar (=)	...	
cuadrado ()	sino	
raizCuadrada ()	<acciones>.	

Nótese que las operaciones de la **tabla 1.1** se agrupan en tres categorías: **secuencia**, **decisión** e **iteración**. Estas categorías se llaman **estructuras de control**. La estructura de control de **secuencia** incluye operaciones que producen una acción o resultado único. Sólo se proporciona una lista parcial de las operaciones de secuencia. Esta lista se incrementará conforme se necesiten operaciones adicionales. Como su nombre lo indica, la estructura de control de **decisiones** incluye las operaciones que le permiten a la computadora tomar decisiones. Por último, la estructura de control de **iteración** incluye aquellas operaciones que se usan para ciclos o repeticiones dentro del algoritmo. Muchas de las operaciones listadas en la **tabla 1.1** se explican por si mismas. Las que no, se explicarán en detalle conforme empecemos a desarrollar algoritmos más complejos.

Para indicar el rango de las instrucciones de decisión y las iterativas, puede o bien utilizarse sangrías o algún tipo de signos o palabras de agrupación como **inicio** y **fin**.

A continuación elaboraremos una serie de algoritmos que resuelvan los problemas planteados.

Ejemplo 1.1**Ir al cine.**

Para solucionar este problema, se debe seleccionar una película de la cartelera del periódico, ir a la sala y comprar la entrada para, finalmente, poder ver la película. Lo que nos lleva a la siguiente solución:

Algoritmo **irAlCine()**

INICIO

// Seleccionar la película

Tomar el periódico.

*Mientras no lleguemos a la cartelera
pasar la hoja.*

Mientras no se acabe la cartelera

Inicio

leer película.

Si nos gusta, anotarla.

Fin.

Elegir una de las películas seleccionadas.

Leer la dirección de la sala y la hora de proyección.

// Comprar la entrada

Trasladarse a la sala.

Si hay entrada

Inicio

Si hay cola

Inicio

Colocarse al final de la cola.

Mientras no lleguemos a la taquilla

Avanzar.

Fin.

Comprar la entrada.

// Ver la película

Leer el número de asiento de la entrada.

Buscar el asiento.

Sentarse.

Ver la película.

Fin.

FIN.

Ejemplo 1.2**Comprar una entrada para ir a los toros.**

Hay que ir a la taquilla y elegir la entrada deseada. Si hay entradas se compra la entrada deseada. Si no la hay, se puede seleccionar otro tipo de entrada o desistir, repitiendo esta acción hasta que se ha conseguido la entrada o el posible comprador ha desistido u opta por comprar en la reventa.

Algoritmo **irALosToros()**

INICIO

Ir a la taquilla.

Si hay cola

Inicio

Colocarse al final de la cola.

Mientras no se llegue a la taquilla

Avanzar.

Fin.

Si hay entrada en taquilla

```

Inicio
    // Comprar la entrada
    Mientras no hay la selección deseada o se terminen las opciones
        Inicio
            Seleccionar sol o sombra.
            Seleccionar barrera, tendido, andanada o palco.
            Seleccionar número de asiento.
            Solicitar la entrada.
            Si hay la entrada solicitada.
                Adquirir la entrada.
            Fin.
        Fin.
    Si no
        Inicio
            Si nos interesa comprar en la reventa
                Inicio
                    Ir con los revendedores.
                    Solicitar que tipo de boletos tienen.
                    Si nos interesa el tipo de boletos
                        Adquirir la entrada.
                    Fin.
                Fin.
            Fin.
        FIN.

```

Ejemplo 1.3

Poner la mesa para la comida.

Para poner la mesa, después de poner el mantel, se toman las servilletas hasta que su número coincide con el de comensales y se colocan. La operación se repetirá con los vasos, platos y cubiertos.

Algoritmo ponerMesa()

```

INICIO
    Poner el mantel.
    Hacer
        Tomar una servilleta y colocarla.
    Mientras el número de servilletas no sea igual al de comensales.

    Hacer
        Tomar un vaso y colocarlo.
    Mientras el número de vasos no sea igual al de comensales.

    Hacer
        Tomar un juego de platos y colocarlo.
    Mientras el número de juegos de platos no sea igual al de comensales.

    Hacer
        Tomar un juego de cubiertos y colocarlo.
    Mientras el número de juegos de cubiertos no sea igual al de comensales.
FIN.

```

Ejemplo 1.4

Hacer una taza de té.

Después de echar agua en la tetera, se pone al fuego y se espera a que el agua hierva (hasta que suena el pitido de la tetera). Introducimos el té y se deja un tiempo hasta que está hecho.

```

Algoritmo hacerTe()
  INICIO
    Tomar la tetera.
    Llenarla de agua.
    Encender el fuego.
    Poner la tetera en el fuego.
    Mientras no hierva el agua
      Esperar.
    Tomar la bolsa de té.
    Introducir en la tetera.

    Mientras no esté hecho el té
      Esperar.
    Echar el té en la taza.
  FIN.

```

Ejemplo 1.5

Lavar los platos de la comida.

Para fregar los platos: abrir la llave de agua, limpiar los platos con un estropajo con jabón, colocarlos en el escurridor y finalmente secarlos.

```

Algoritmo lavarPlatos()
  INICIO
    Abrir la llave de agua.
    Tomar el estropajo.
    Echarle jabón.
    Mientras queden platos
      Inicio
        Lavar el plato.
        Dejarlo en el escurridor.
      Fin.
    Mientras queden platos en el escurridor
      Secar plato.
  FIN.

```

Ejemplo 1.6

Reparar un pinchazo en la rueda de una bicicleta.

Después de desmontar la rueda y la cubierta e inflar la cámara, se introduce la cámara por secciones en un cubo de agua. Las burbujas de aire indicarán donde está el pinchazo. Una vez descubierto el pinchazo se aplica el pegamento y ponemos el parche. Finalmente se montan la cámara, la cubierta y la rueda.

```

Algoritmo repararPonchadura()
  INICIO
    Desmontar la rueda.
    Desmontar la cubierta.
    Sacar la cámara.
    Inflar la cámara.
    Meter una sección de la cámara en un cubo de agua.
    Mientras no salgan burbujas o se hallan examinado todas las secciones de la cámara
      Meter una sección de la cámara en un cubo de agua.
    Si salen burbujas
      Inicio
        Marcar el pinchazo.
        Echar pegamento.
        Mientras no este seco
          Esperar.
        Poner el parche.
      Fin.
  FIN.

```

Mientras no este fijo
 Oprimir.
 Montar la cámara.
 Montar la cubierta.
 Montar la rueda.

Fin.

FIN.

Ejemplo 1.7

Pagar una multa de tránsito.

Debe elegir cómo desea pagar la multa, si en efectivo en tesorería o por medio de una transferencia bancaria. Si elige el primer caso, tiene que ir a tesorería, desplazarse a la ventanilla de pago de multas, pagarla en efectivo y recoger el comprobante de pago.

Si desea pagarla por transferencia bancaria, hay que ir al banco, llenar el impreso apropiado, dirigirse a la ventanilla, entregar el impreso, el pago y recoger el comprobante de pago.

Algoritmo *multaTransito()*

INICIO

Si pagamos en efectivo

Inicio

Ir a tesorería.

Ir a la ventanilla de pago de multas.

Si hay cola

Inicio

Colocarse al final de la cola.

Mientras no lleguemos a la ventanilla
 avanzar.

Fin.

Entregar la multa.

Entregar el dinero.

Recoger el comprobante de pago.

Fin.

Si no

// Pagamos por transferencia bancaria

Inicio

Ir al banco.

llenar el formato.

Si hay cola

Inicio

Ponerse al último.

Mientras no lleguemos a la ventanilla
 Avanzar.

Fin.

Entregar el impreso.

Recoger el comprobante de pago.

Fin.

FIN.

Ejemplo 1.8

Hacer una llamada telefónica. Considerar los casos: **a) llamada manual con operador, b) llamada automática, c) llamada por cobrar.**

Para decidir el tipo de llamada que se efectuará, primero se debe considerar si se dispone de efectivo o no (para realizar la llamada por cobrar) Si hay efectivo se debe ver si el lugar donde vamos a llamar está conectado a la red automática o no.

Para una llamada con operadora hay que llamar a la central y solicitar la llamada, esperando hasta que se establezca la comunicación. Para una llamada automática se leen los prefijos del país y estado si fuera necesario y se realiza la llamada, esperando hasta que contesten. Para llamar por cobrar se debe llamar a la central, solicitar la llamada y esperar a que la persona con quien se desea comunicar autorice la llamada, con lo que se establecerá la comunicación.

Algoritmo **llamadaTelefonica()**

INICIO

Si tenemos tarjeta con suficiente dinero

Si podemos hacer una llamada automática

Inicio

Leer el prefijo del país y localidad.

Marcar el número.

Fin.

Si no

Inicio

// llamada manual

Llamar a la central.

Solicitar la comunicación.

Fin.

Mientras no contesten

Esperar.

Establecer comunicación.

Si no

Inicio

// realizar una llamada por cobrar

Llamar a la central.

Solicitar la llamada.

Esperar hasta tener la autorización.

Establecer comunicación.

Fin.

FIN.

Ejemplo 1.9

Cambiar el cristal roto de una ventana.

Algoritmo **cambiarVidrio()**

INICIO

Para $i = 1$ hasta $i = 4$ hacer en pasos de 1

Inicio

Quitar un tornillo.

Mientras el número de tornillos quitados no sea igual al total de tornillos

Quitar un tornillo.

Sacar la moldura.

Fin.

Sacar el cristal roto.

Poner el cristal nuevo.

Para $i = 1$ hasta $i = 4$ hacer en paso de 1

Inicio

Colocar la moldura.

Poner un tornillo.

Mientras el número de tornillos puestos no sea igual al total de tornillos

Poner un tornillo.

Fin.

FIN.

Ejemplo 1.10**Realizar una llamada telefónica desde un teléfono público.**

Se debe de ir a la cabina y esperar si hay cola. Entrar e introducir la tarjeta. Se marca el número y se espera la señal, si está ocupado o no contestan se repite la operación hasta que descuelguen el teléfono o decida irse.

Algoritmo **llamadaTelefonica()**

INICIO

Ir a la cabina.

Mientras haya cola

Avanzar.

Entrar en la cabina.

Introducir la tarjeta.

Hacer

Inicio

Marcar el número.

Si contestan antes de 10 timbrazos

Hablar.

Si no

Colgar.

Fin.

Mientras desee marcar nuevamente.

FIN.

Ejemplo 1.11

Averiguar si una palabra es un palíndromo. Un **palíndromo** es una palabra que se lee igual de izquierda a derecha que de derecha a izquierda, como por ejemplo: radar.

Para comprobar si una palabra es un palíndromo, se puede ir formando una palabra con los caracteres invertidos con respecto al original y comprobar si la palabra al revés es igual a la original. Para obtener esa palabra al revés se leerán en sentido inverso los caracteres de la palabra inicial y se irán juntando sucesivamente hasta llegar al primer carácter.

Algoritmo **palindromo()**

INICIO

Leer(palabra).

Leer(último carácter).

Mientras halla caracteres

Inicio

Juntar el carácter a los anteriores.

Leer(carácter anterior).

Fin.

Si las dos palabras son iguales

Escribir("Es un palíndromo").

Si no

Escribir("No es un palíndromo").

FIN.

Ejemplo 1.12

Escribir un algoritmo para determinar el máximo común divisor de dos números enteros utilizando el algoritmo de Euclides.

Para hallar el máximo común divisor de dos números se debe dividir uno entre otro. Si la división es exacta, es decir si el residuo es 0, el máximo común divisor es el divisor. Si no, se deben dividir otra

vez los números, pero en este caso el dividendo será el antiguo divisor y el divisor el residuo de la división anterior. El proceso se repetirá hasta que la división sea exacta.

Algoritmo **maximoDivisor()**

INICIO

Leer(a,b).

Mientras $a \bmod b \neq 0$

Inicio

residuo = $a \bmod b$.

$a = b$.

$b = \text{residuo}$.

Fin.

$\text{mcd} = b$.

Escribir(mcd).

FIN.

Ejemplo 1.13

Diseñar un algoritmo que lea e imprima una serie de números distintos de cero. El algoritmo debe terminar con un valor cero que no se debe imprimir. Finalmente se desea obtener la cantidad de valores leídos distintos de 0.

Se deben leer números dentro de un ciclo que terminará cuando el último número leído sea cero. Cada vez que ejecute dicho ciclo y antes que se lea el siguiente número se imprime éste y se incrementa el contador en una unidad. Una vez se haya salido del ciclo se debe escribir la cantidad de números leídos, es decir, el contador.

Algoritmo **imprimirNumeros()**

INICIO

contador = 0.

Leer(número).

Mientras número sea distinto de cero

Inicio

Escribir(número).

Incrementar contador en 1.

Leer(número).

Fin.

Escribir(contador).

FIN.

Ejemplo 1.14

Diseñar un algoritmo que imprima y sume la serie de números 3, 6, 9, 12, ..., 99.

Se trata de idear un método con el que obtengamos dicha serie, que no es más que incrementar una variable de tres en tres. Para ello se hará un ciclo que acaba cuando el número sea mayor que 99 (o cuando se realice 33 veces). Dentro de este ciclo se incrementa la variable, se imprime y se acumula su valor en otra variable llamada **suma**, que será el dato de salida.

Algoritmo **imprimeAcumula()**

INICIO

suma = 0.

número = 3.

Mientras número ≤ 99

Inicio

Escribir(número).

suma = suma + número.

número = número + 3.

Fin.

Escribir(suma).

FIN.

Ejemplo 1.15

Escribir un algoritmo que lea cuatro números y a continuación escriba el mayor de los cuatro.

Hay que comparar los cuatro números, pero no hay necesidad de compararlos todos con todos. Por ejemplo, si a es mayor que b y a es mayor que c, es evidente que ni b ni c son los mayores, por lo que si a es mayor que d el número mayor será a y en caso contrario lo será d.

```

Algoritmo mayorDeCuatroNumeros()
  INICIO
    Leer(a, b, c, d).
    Si a > b
      Si a > c
        Si a > d
          mayor = a.
        Si no
          mayor = d.
      Si no
        Si c > d
          mayor = c.
        Si no
          mayor = d.
    Si no
      Si b > c
        Si b > d
          mayor = b.
        Si no
          mayor = d.
      Si no
        Si c > d
          mayor = c.
        Si no
          mayor = d.
    Escribir(mayor).
  FIN.

```

Ejemplo 1.16

Diseñar un algoritmo para calcular la velocidad (en metros / segundo) de los corredores de una carrera de 1500 metros. Las entradas serán parejas de números (minutos, segundos) que darán el tiempo de cada corredor. Por cada corredor se imprimirá el tiempo en minutos y segundos, así como la velocidad media. El ciclo se ejecutará hasta que demos una entrada de 0, 0 que será la marca de fin de entrada de datos.

*Se debe efectuar un ciclo que se ejecute hasta que mm sea 0 y ss sea 0. Dentro del bucle se calcula el tiempo en segundos con la fórmula $\text{tiempo} = \text{ss} + \text{mm} * 60$. La velocidad se hallará con la fórmula $\text{velocidad} = \text{distancia} / \text{tiempo}$.*

```

Algoritmo velocidad()
  INICIO
    distancia = 1500.
    Leer(mm, ss).
    Mientras (mm != 0 o ss != 0)
      Inicio
        tiempo = ss + mm * 60.
        velocidad = distancia / tiempo.
        Escribir(mm, ss, velocidad).
        Leer(mm, ss).
      Fin.
    FIN.

```

Ejemplo 1.17

Diseñar un algoritmo para determinar si un número n es primo. (Un número primo sólo es divisible por el mismo y por la unidad)

Una forma de averiguar si un número es primo es por tanteo. Para ello se divide sucesivamente el número por los números comprendidos entre 2 y n . Si antes de llegar a n encuentra un divisor exacto, el número no será primo. Si el primer divisor es n el número será primo.

Por lo tanto se hará un bucle en el que una variable (**divisor**) irá incrementándose en una unidad entre 2 y n . El bucle se ejecutará hasta que se encuentra un divisor, es decir hasta que $n \bmod \text{divisor} = 0$. Si al salir del bucle **divisor** = n , el número será primo.

```

Algoritmo primo()
  INICIO
    Leer ( $n$ ).
    Asignar 2 a divisor.
    Mientras  $n \bmod \text{divisor} \neq 0$ 
      Incrementar divisor en 1.
    Si divisor =  $n$ 
      Escribir("Es primo.").
    Si no
      Escribir("No es primo.").
  FIN.

```

Ejemplo 1.18

Escribir un algoritmo que calcule la superficie de un triángulo en función de la base y la altura.

Para calcular la superficie se aplica la fórmula $\text{superficie} = \text{base} * \text{altura} / 2$.

```

Algoritmo superficieTriangulo()
  INICIO
    Leer(base, altura).
     $\text{superficie} = \text{base} * \text{altura} / 2$ .
    Escribir(superficie).
  FIN.

```

EXAMEN BREVE 1-2**ABSTRACCIÓN DE PROBLEMAS Y REFINAMIENTO SUCESIVO**

Los seres humanos se han convertido en la especie más influyente de este planeta, debido a su capacidad para **abstraer el pensamiento**. Los sistemas complejos, sean naturales o artificiales, sólo pueden ser comprendidos y gestionados cuando se omiten detalles que son irrelevantes a nuestras necesidades inmediatas. El proceso de excluir detalles no deseados o no significativos, al problema que se trata de resolver, se denomina **abstracción**, y es algo que se hace en cualquier momento.

Cualquier sistema suficientemente complejo se puede visualizar en diversos **niveles de abstracción** dependiendo del propósito del problema. Si nuestra intención es conseguir una visión general del proceso, las características del proceso presente en nuestra abstracción constará principalmente de generalizaciones. Sin embargo, si se trata de modificar partes de un sistema, se necesitará examinar esas partes con mayor nivel de detalle. Cons-

deremos el problema de representar un sistema relativamente complejo tal como un coche. El nivel de abstracción será diferente según sea la persona o entidad que se relaciona con el coche: conductor, propietario, mecánico o fabricante.

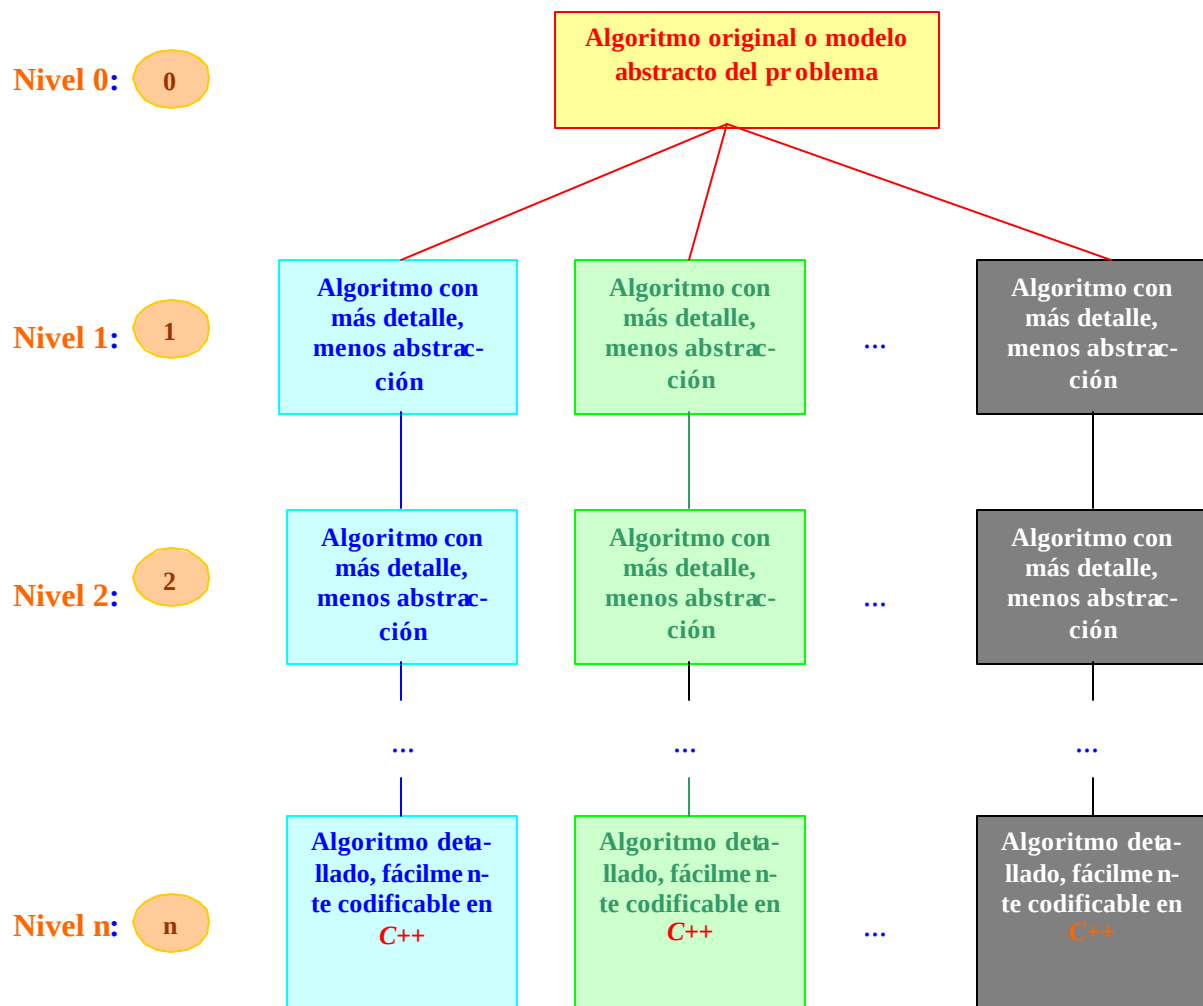
Así, desde el punto de vista del conductor sus características se expresan en términos de sus funciones (acelerar, frenar, conducir, etc.); desde el punto de vista del propietario sus características se expresan en función de nombre, dirección, edad; la mecánica del coche es una colección de partes que cooperan entre sí para proveer las funciones citadas, mientras que desde el punto de vista del fabricante interesa su precio, producción anual de la empresa, duración de construcción, etc. La existencia de diferentes niveles de abstracción conduce a la idea de una *jerarquía de abstracciones*.

Las soluciones a problemas no triviales tiene una jerarquía de abstracciones de modo que sólo los objetivos generales son evidentes al nivel más alto. A medida que se desciende en nivel los aspectos diferentes de la solución se hacen evidentes.

De esta manera, para resolver un problema, primero necesita obtener la *imagen completa* del mismo. Una vez que tiene la gran imagen de la solución del problema, puede en forma gradual *refinar* la solución proporcionando más detalles hasta que tenga una solución, la cual se codificará fácilmente en un lenguaje de computadora. El proceso de adicionar poco a poco más detalle a una solución del problema general se conoce con el nombre de *refinamiento sucesivo* o *refinamiento paso a paso*.

Como ejemplo, considere el problema de diseñar la *casa de sus sueños*. ¿Deberá empezar por dibujar los planos detallados de la casa? Probablemente no, porque es muy posible que se pierda en los detalles. Un mejor método sería hacer primero una perspectiva general de la casa. Después elaborar un diagrama de la planta general y por último, dibujar los detalles de la construcción de la casa, a la cual deberán apegarse los constructores.

Los conceptos de *abstracción del problema* y *refinamiento sucesivo* le permiten *dividir* y *vencer* el problema y solucionarlo de *arriba-abajo (top-down)* o *en forma descendente*. Esta estrategia ha sido probada para solucionar todo tipo de problemas, en especial problemas de programación. En programación se genera una solución general del problema o algoritmo y en forma gradual lo refinamos, produciendo algoritmos más detallados, hasta que llegamos a un nivel que puede ser fácilmente codificado usando un lenguaje de programación. Esta idea se ilustra en la *figura 1.2*.



- 0 **Algoritmo original o modelo abstracto del problema.**
- 1 **Primer nivel** de refinamiento mostrando más detalles del problema por medio de algoritmos adicionales.
- 2 **Segundo nivel** de refinamiento mostrando más detalles del problema por medio de algoritmos adicionales.
- n **Enésimo nivel final de refinamiento.** Los algoritmos están en forma codificable.

*Figura 1.2. La solución del problema empieza con un **modelo abstracto** general del problema, que es **refinado paso a paso**, produciendo más y más detalle, hasta que se alcanza un nivel de **algoritmos codificables**.*

En resumen, cuando se resuelve un problema, siempre se comienza con una imagen global, se empieza con un modelo abstracto o general de la solución. Esto le permite con-

centrarse en el problema sin perderse en los detalles acerca de la implementación a un lenguaje de programación en particular. Posteriormente, en forma gradual se refina la solución hasta alcanzar un nivel que pueda ser fácilmente codificado usando un lenguaje de programación estructurada, como **C++**. Los ejemplos de *Solución de problemas en acción* que se muestran a continuación ilustran este proceso.

EXAMEN BREVE 1-3

SOLUCIÓN DE PROBLEMAS EN ACCIÓN: **TEOREMA DE PITÁGORAS**

PROBLEMA

Desarrolle una serie de algoritmos para encontrar el valor de la hipotenusa de un triángulo rectángulo, conocidos sus dos catetos, use el teorema de Pitágoras. Construya los algoritmos utilizando las instrucciones algorítmicas de la tabla 1.1.

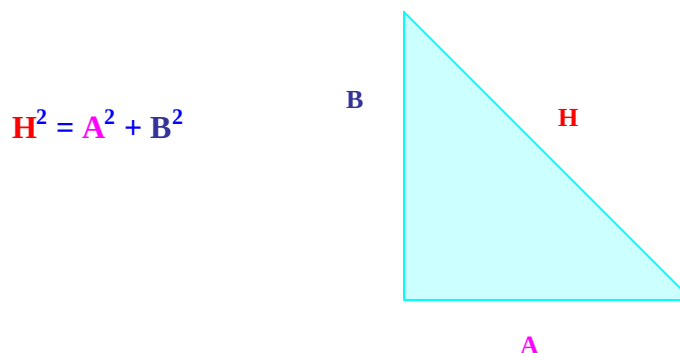


Figura 1.3. Cálculo de la hipotenusa de un triángulo rectángulo mediante el teorema de Pitágoras.

DEFINICIÓN DEL PROBLEMA

Cuando defina el problema, debe considerar tres elementos: **salida**, **entrada** y **procesamiento** relacionados con el enunciado del problema. Llamemos a los dos catetos **A** y **B** y a la hipotenusa **H**. El problema requiere que encontremos el valor de la hipotenusa (**H**). Mostraremos el valor de ésta en la pantalla del monitor. Para obtener este resultado, los dos catetos (**A** y **B**) deben ser recibidos por el programa. Vamos a suponer que el usuario debe ingresar estos valores por medio del teclado.

El teorema de Pitágoras nos dice que el cuadrado de la hipotenusa es igual a la suma de los cuadrados de los dos catetos. En símbolos:

$$H^2 = A^2 + B^2$$

Esta ecuación representa el proceso que debe realizar la computadora. En resumen, la definición del problema es como sigue:

Salida:

Este programa calcula el valor de la **hipotenusa** de un triángulo rectángulo, dados los catetos **A** y **B**.

Favor de introducir el valor del primer **cateto** : **3**

Favor de introducir el valor del segundo **cateto** : **4**

	<i>El valor de la hipotenusa es: 5</i>
Entrada:	<i>Los valores, que serán ingresados por el usuario por medio del teclado, de los dos catetos (A y B)</i>
Procesamiento:	<i>Empleo del teorema de Pitágoras: $H^2 = A^2 + B^2$.</i>

Ahora que el problema se ha definido en términos de salida, entrada y procesamiento es tiempo de planear la solución desarrollando los algoritmos necesarios.

PLANEACIÓN DE LA SOLUCIÓN

Empezaremos con un modelo abstracto del problema. La ilustración en la **figura 1.4** se denomina **diagrama de estructura**, porque presenta la estructura general de la solución de nuestro problema. El diagrama de estructura muestra cómo se ha dividido el problema en una serie de subproblemas, cuya solución conjunta resolverá la situación inicial. Con el uso del diagrama de estructura y los algoritmos, que se darán más adelante, para resolver este problema, puede ser codificado con facilidad en cualquier lenguaje estructurado, como **C++**. Ciertamente el ejemplo es muy sencillo, pero muestra los conceptos de diseño estructurado de arriba-abajo usando el refinamiento sucesivo.

Observe que la idea es básicamente dividir el problema (*obtener el valor de la hipotenusa de un triángulo rectángulo, dados el valor de sus dos catetos*) en tres subproblemas:

- *Obtener del usuario el valor de cada uno de los catetos.*
- *Calcular el valor de la hipotenusa utilizando el teorema de Pitágoras.*
- *Mostrar en la pantalla del monitor el valor de la hipotenusa.*

Veamos en forma gráfica estas ideas:

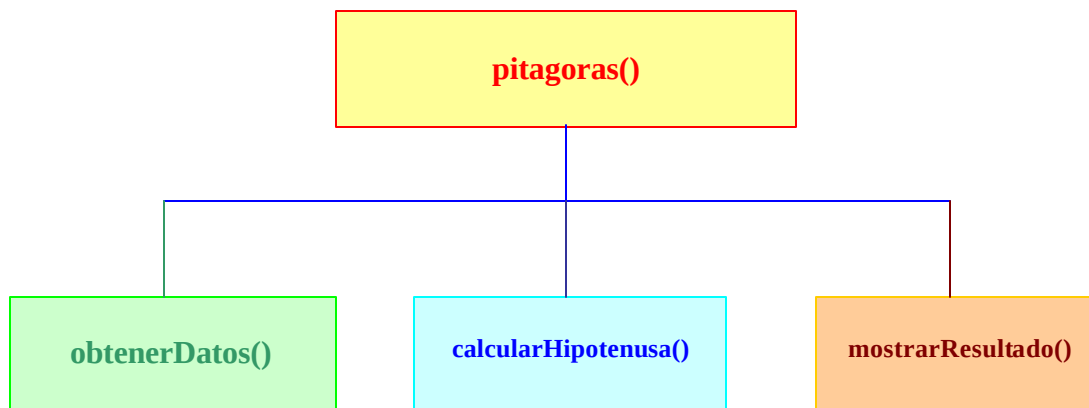


Figura 1.4. *Diagrama de estructura para el problema del teorema de Pitágoras. Muestra el diseño estructurado de arriba-abajo necesario para solucionar el problema usando un lenguaje estructurado como **C++**.*

Desde el punto de vista de algoritmos la solución al problema será la siguiente:

Empezaremos con un modelo abstracto del problema. Este será nuestro algoritmo inicial, al cual nos referiremos con el nombre de **pitagoras()** (el nombre es arbitrario) En este nivel sólo nos interesan las directrices de las operaciones principales que se requieren para la solución. Estas se derivan directamente de la definición del problema. Como resultado, nuestro algoritmo inicial es como sigue:

ALGORITMO INICIAL

```

pitagoras()
  INICIO
    Llamar a la función obtenerDatos()
    Llamar a la función calcularHipotenusa()
    Llamar a la función mostrarResultado()
  FIN.

```

Note que en este nivel no interesa la forma en que se van a realizar las operaciones anteriores en un lenguaje de computadora. Sólo son de interés las principales operaciones del programa, sin preocuparse por los detalles de la implementación del lenguaje ¡Esto es la abstracción del problema!

El siguiente paso es refinar en forma sucesiva el algoritmo inicial hasta que obtengamos uno o más algoritmos que puedan ser codificados en forma directa en algún lenguaje. Este es un problema relativamente simple, así que podemos emplear las operaciones de pseudo código listadas en la **tabla 1.1** en el primer nivel de refinamiento. En el algoritmo inicial anterior se identifican tres operaciones principales: **la obtención de los datos introducidos por el usuario, el cálculo de la hipotenusa y la muestra de los resultados al usuario**. Crearemos tres algoritmos adicionales que implementen estas operaciones. Primero, la introducción de los datos por el usuario. Llamaremos a este algoritmo **obtenerDatos()**

PRIMER NIVEL DE REFINAMIENTO

```

obtenerDatos()
  Inicio
    Escribir("Este programa calcula el valor de la hipotenusa de un triángulo rec-
      tángulo, dados los catetos A y B").
    Escribir("Favor de introducir el valor del primer cateto: ").
    Leer (A).
    Escribir("Favor de introducir el valor del segundo cateto: ").
    Leer (B).
  Fin.

```

La siguiente tarea es desarrollar un algoritmo para calcular la hipotenusa del triángulo. Llamaremos a este algoritmo **calcularHipotenusa()** y emplearemos las operaciones de pseudo código necesarias, según la **tabla 1.1**.

```

calcularHipotenusa()
  Inicio
    Asignar (cuadrado(A)) a x.
    Asignar (cuadrado(B)) a y.
    Asignar (x + y) a H.
    Asignar (raízCuadrada(H)) a H.
  Fin.

```

La tarea final es desarrollar un algoritmo para mostrar los resultados. Llamaremos a este algoritmo **mostrarResultado()**. Todo lo que necesitamos aquí es una operación de *Escribir* como sigue:

```

mostrarResultado()
  Inicio
    Escribir ("El valor de la hipotenusa es: ", H).
  Fin.

```

Eso es todo.

Cuando empiece a codificar en **C++**, traduciré los algoritmos anteriores directamente al código de **C++**. Cada uno de los algoritmos será codificado como una **función C++**. Las funciones en **C++** son

subprogramas diseñados para realizar tareas específicas, como aquellas realizadas por cada uno de nuestros algoritmos. Más aún, codificaremos una función llamada **obtenerDatos()** para obtener los datos del usuario, otra función llamada **calcularHipotenusa()** para calcular la hipotenusa y una tercera función llamada **mostrarResultado()** para mostrar los resultados finales al usuario. Además, codificaremos una función llamada **pitagoras()** para llamar a estas funciones en forma secuencial.

SOLUCIÓN DE PROBLEMAS EN ACCIÓN: IMPUESTO DE LAS VENTAS

PROBLEMA

Desarrolle una serie de algoritmos para calcular el **importe del impuesto** sobre las ventas y el **costo total** (incluye el importe del impuesto) de los artículos vendidos. Supongamos que la **tasa del impuesto** sobre las ventas es del 7% y que el usuario ingresará el **costo** del artículo.

DEFINICIÓN DEL PROBLEMA

Busque los *nombres* y *verbos* dentro de la declaración del problema, a menudo proporcionan pistas para la salida, entrada y procesamiento requeridos. Los nombres sugieren salida y entrada; los verbos procesamiento. Los nombres relativos a la salida y la entrada son *importe del impuesto*, *costo total*, *tasa del impuesto* y *costo*. El *costo total* es la salida requerida y la *tasa del impuesto* y *costo* por artículo son necesarios como entrada para calcular el *costo total* del artículo. Sin embargo, la *tasa del impuesto* sobre las ventas es determinada (7%), así que el único dato requerido por el algoritmo es que el usuario introduzca el *costo* del artículo.

El verbo *calcular* nos sugiere dos cosas para procesar: el *impuesto* sobre las ventas y el *costo total* de los artículos (incluido el impuesto sobre las ventas). Por lo tanto, el procesamiento debe calcular el importe del impuesto sobre la venta y sumar este valor al costo del artículo para obtener el costo total del artículo. En resumen, la definición del problema en términos de salida, entrada y procesamiento es como sigue:

Salida:	Este programa calcula el impuesto que se paga por un artículo, así como el costo total del mismo. Favor de introducir el costo del artículo: El costo total es de:
Entrada:	El costo del artículo vendido será ingresado por el usuario por medio del teclado.
Procesamiento:	Impuesto = $0.07 \times \text{Costo}$ Costo total = $\text{Costo} + \text{Impuesto}$.

PLANEACIÓN DE LA SOLUCIÓN

Usando la definición del problema anterior, ahora estamos listos para escribir el algoritmo inicial como sigue:

ALGORITMO INICIAL

```

impuestoVentas()
  INICIO
    Llamar a la función obtenerDato().
    Llamar a la función calcularCostoTotal().
    Llamar a la función mostrarResultado().
  FIN.

```

Otra vez, hemos dividido el problema en tres tareas principales relacionando la salida, entrada, y procesamiento. La siguiente tarea es escribir un algoritmo en pseudo código para cada una de las tareas.

Nos referiremos a estos algoritmos como **obtenerDato()**, **calcularCostoTotal()** y **mostrarResultado()**. Debido a la simplicidad de este problema, sólo necesitamos un nivel de refinamiento.

PRIMER NIVEL DE REFINAMIENTO

obtenerDato()

Inicio

Escribir (“Este programa calcula el impuesto que se paga por un artículo, así como el costo total del mismo”).

Escribir (“Favor de introducir el costo del artículo: ”).

Leer (Costo).

Fin.

calcularCostoTotal()

Inicio

Asignar ($0.07 \times \text{Costo}$) a *Impuesto*

Asignar ($\text{Costo} + \text{Impuesto}$) a *costoTotal*.

Fin.

mostrarResultado()

Inicio

Escribir (“El costo total es de: ”, *costoTotal*)

Fin.

¿Puede desarrollar un **diagrama de estructura** para esta solución?

SOLUCIÓN DE PROBLEMAS EN ACCIÓN: INTERÉS DE UNA TARJETA DE CRÉDITO

PROBLEMA

Supongamos que el **interés** que se carga a una cuenta de tarjeta de crédito se calcula de acuerdo con el siguiente criterio: el interés cargado es **18%** para los primeros **\$500.00** y **15%** para cualquier cantidad arriba de **\$500.00**. Desarrolle los algoritmos requeridos para encontrar el **importe total del interés** dado el balance o estado de una cuenta.

Comencemos por definir el problema en cuanto a salida, entrada y procesamiento.

DEFINICIÓN DEL PROBLEMA

Salida:

Este programa calcula los **intereses** de una tarjeta de crédito. Los primeros \$500.00 del estado actual de la deuda pagan un interés del 18%. Si la cuenta excede los \$500.00, el impuesto es del 15% sobre lo que exceda a \$500.00

Favor de introducir el **estado de la cuenta**:

El importe del **interés** es:

Entrada:

Supondremos que el usuario ingresará el importe del **balance** por medio del teclado.

Procesamiento:

Esta es una aplicación en la cual debe incluirse una operación de toma de decisiones en el algoritmo. Hay dos posibilidades que son como siguen:

1. Si el **balance** es menor o igual a \$500.00 entonces el **interés** es 18% del **balance**, o

$$\text{Interés} = 0.18 \cdot \text{Balance}$$

2. Si el **balance** es superior a \$500.00, entonces el **interés** es 18% de los primeros \$500.00 más 15% de cualquier cantidad arriba de \$500.00. En forma de ecuación:

$$\text{Interés} = (0.18 \cdot 500) + [0.15(\text{Balance} - 500)]$$

Note el uso de las dos declaraciones **si** condición **entonces** en estas dos posibilidades.

PLANEACIÓN DE LA SOLUCIÓN

Nuestra solución del problema empieza con el algoritmo inicial que hemos llamado **interesTarjeta()**. Este algoritmo simplemente reflejará la definición del problema :

ALGORITMO INICIAL

```

interesTarjeta()
  INICIO
    Llamar a la función obtenerDato().
    Llamar a la función calcularInteres().
    Llamar a la función mostrarResultado().
  FIN.

```

Debido a la simplicidad del problema sólo es necesario un nivel de refinamiento como sigue:

PRIMER NIVEL DE REFINAMIENTO:

```

obtenerDato()
  Inicio
    Escribir("Este programa calcula los intereses de una tarjeta de crédito. Los pri-
      meros $500.00 del estado actual de la deuda pagan un interés del 18%. Si
      la cuenta excede los $500.00, el impuesto es del 15% sobre lo que exceda
      a $500.00").
    Escribir("Favor de introducir el estado de la cuenta: ").
    Leer (Balance).
  Fin.

calcularInteres()
  Inicio
    Si Balance <= 500 entonces
      Asigna (0.18 · Balance) a Interés.
    Si Balance > 500 entonces
      Asigna [(0.18 · 500) + 0.15(Balance – 500)] a Interés.
  Fin.

mostrarResultado()
  Inicio
    Escribir ("El importe del interés es: ", Interés)
  Fin.

```

¿Puede desarrollar un **diagrama de estructura** para esta solución?

LO QUE NECESITA SABER

Antes de continuar con la siguiente lección, asegúrese de haber comprendido los siguientes conceptos:

- ❑ Los cinco pasos principales que deben realizarse cuando desarrolle aplicaciones son: (1) **definir** el problema, (2) **planear** la solución del problema, (3) **codificar** el programa, (4) **verificar** y **depurar** el programa y (5) **documentar** el programa.
- ❑ Cuando defina el problema, considere los requerimientos de **salida**, **entrada** y **procesamiento** de la aplicación.
- ❑ La planeación de la solución del problema requiere que especifique los pasos de la solución del problema mediante un **algoritmo**.
- ❑ Un **algoritmo** es una serie de instrucciones paso a paso que proporcionan una solución al problema en una cantidad finita de tiempo.
- ❑ La **abstracción** y el **refinamiento** sucesivo son herramientas poderosas para la solución de problemas.
- ❑ La **abstracción** le permite ver el problema en términos generales, sin preocuparse por los detalles de la implementación en un lenguaje de computadora.
- ❑ El **refinamiento** sucesivo se aplica a una solución inicial abstracta del problema, posteriormente desarrollar de manera gradual una serie de algoritmos relacionados que pueden ser directamente codificados usando un lenguaje estructurado, como **C++**.
- ❑ Una vez que se desarrolla una serie de **algoritmos**, deben de ser **codificados** en algún lenguaje formal.
- ❑ El lenguaje que se utiliza en estas lecciones es **C++**.
- ❑ Una vez **codificado**, el programa debe ser **verificado** y **depurado** a través de la prueba de escritorio, compilación y ejecución.
- ❑ Por último el proceso de programación, desde la definición del problema hasta la verificación y depuración, debe ser **documentado** para que pueda ser fácilmente entendido por usted o por cualquiera que trabaje con él.
- ❑ Su **depurador C++** es una de las mejores herramientas que puede utilizar para tratar con fallas que surgen dentro de sus programas. Le permite hacer depuración a nivel fuente y le ayuda con las dos partes más difíciles de la depuración: encontrar el error y encontrar la causa del error.
- ❑ El **depurador** le permite rastrear una a una sus funciones dentro de sus programas. El uso de un depurador hace más lenta la ejecución del programa, ya que le permite examinar el contenido de los elementos de datos individuales y la salida del programa en cualquier punto dado del programa.

PREGUNTAS Y PROBLEMAS

PREGUNTAS

1. Defina lo que se entiende por **algoritmo**.
2. Liste los cinco pasos del **algoritmo** del programador.
3. ¿Cuáles son los tres **hechos** que debe considerar durante la fase de **definición del problema**?
4. ¿Qué **herramientas** se emplean para **planear las soluciones** de un problema de programación?
5. Explique cómo la **abstracción** ayuda en la solución de los problemas.
6. Explique el proceso de **refinamiento sucesivo**.
7. La **escritura** de un programa se conoce con el nombre de: _____.
8. Determine tres cosas que puede hacer para **verificar** y **depurar** sus programas.
9. Liste los elementos mínimos requeridos para una buena **documentación**.
10. ¿Cuáles son las tres **características** que un buen **algoritmo** de computadora debe poseer?
11. Las tres principales **estructuras** de control de un **lenguaje de programación estructurado** son _____, _____, _____.
12. Explique por qué una operación **si condición entonces** en el problema de interés de tarjeta de crédito no funcionará. Si sabe que el balance no es menor o igual a \$500, el balance debe ser mayor que \$500. Así que, ¿por qué no es posible eliminar la segunda operación **si condición entonces**?

PROBLEMAS

1. Desarrolle una serie de algoritmos para calcular la **suma**, la **resta**, la **multiplicación** y la **división** de dos enteros ingresados por el usuario.
2. Revise la solución que obtuvo en el **problema 1** para protegerla de un **error de división por cero** en tiempo de ejecución.
3. Desarrolle una serie de algoritmos para leer el total de horas semanales trabajadas por los empleados y el pago por hora. Determine el **pago semanal bruto** usando un pago de vez y media para más de 40 horas.
4. Revise la solución generada en el problema de tarjeta de crédito para emplear una operación **si condición entonces acciones si no acciones** en lugar de las dos operaciones **si condición entonces acciones**.
5. Una dimensión en una parte de un dibujo indica que la longitud de la parte es 3.00 ± 0.25 pulg. Significa que la longitud mínima aceptable de la parte es $3.00 - 0.25 = 2.75$ pulg. y la longitud máxima aceptable de la parte es $3.00 + 0.25 = 3.25$ pulg. Desarrolle una serie de algoritmos que muestren **ACEPTABLE** si la parte está dentro de la tolerancia e **INACEPTABLE** si la parte está fuera de tolerancia. **Muestre** la definición del problema en términos de salida, entrada y procesamiento.
6. Emplee la ley de Ohm para desarrollar una serie de algoritmos para calcular el **voltaje** a partir de los valores de la **corriente** y la **resistencia** ingresados por el usuario. La ley de Ohm determina que el **voltaje** es igual al producto de la **corriente** por la **resistencia**.
7. La **resistencia** de un conductor puede ser calculada basándose en la composición de su material y tamaño usando la siguiente ecuación:

$$R = r (l / A)$$

Donde:

R es la **resistencia** del conductor en Ohm.

r es la **resistividad** del conductor.

l es la **longitud** del conductor, en metros.

A es el **área transversal** del conductor, en metros cuadrados.

*Desarrolle una serie de algoritmos para calcular la **resistencia** de un conductor de cobre de cualquier tamaño, suponga que el usuario ingresa la **longitud** del conductor y el **área** transversal. (Nota: el factor de **resistividad** para el cobre es $1.72 \cdot 10^{-8}$)*

8. **Revise** la solución que se obtuvo del **problema 7**, suponga que el usuario ingresa la **longitud** del conductor en pulgadas y el **área** transversal en pulgadas cuadradas.
9. **Desarrolle** una serie de algoritmos que le permitan la entrada de tres coeficientes enteros de una ecuación cuadrática y calcule las raíces de la ecuación. Proporcione un mensaje de error si existe raíz compleja.
10. **Desarrolle** diagramas de estructura para la solución de los problemas del **impuesto de ventas** y de la **tarjeta de crédito** desarrollados en esta lección.

EXAMEN BREVE 1-1

1. Los **enunciados en castellano** que requieren menos precisión que un lenguaje formal de programación se conoce con el nombre de :_____.
2. ¿Qué preguntas deben contestarse cuando se **define un problema** de programación?
3. ¿Qué puede hacerse para **verificar y depurar** un programa?
4. ¿Por qué es importante **hacer comentarios** dentro de un programa?

EXAMEN BREVE 1-2

1. ¿Por qué es importante **desarrollar un algoritmo** antes de codificar un programa?
2. ¿Cuáles son las tres principales categorías de operaciones del **lenguaje algorítmico**?
3. Mencione tres **operaciones de decisión**.
4. Mencione tres **operaciones de iteración**.

EXAMEN BREVE 1-3

1. Explique por qué la **abstracción** es importante cuando se solucionan problemas.
2. Explique el proceso de **refinamiento sucesivo**.
3. ¿Cómo saber cuándo se ha **alcanzado el nivel de codificación** de un algoritmo cuando se utiliza el refinamiento sucesivo?

RESPUESTAS EXAMEN BREVE 1-1

1. Los **enunciados en castellano** que requieren menos precisión que un lenguaje formal de programación se conoce con el nombre de **pseudo código**.
2. Algunas de las preguntas que deben responderse cuando se define un problema de programación son las siguientes:
 - ¿Qué salidas se requieren?
 - ¿Qué entradas se necesitan?
 - ¿Qué procesamiento se necesita para producir la salida a partir de la entrada?
3. Para **verificar y depurar** un programa: haga una **prueba de escritorio**, compile y depure el programa usando un depurador y ejecútelo.
4. Es importante hacer **comentarios** dentro de un programa, pues éstos explican lo que aquél hace y facilita la lectura y el mantenimiento del mismo.

RESPUESTAS EXAMEN BREVE 1-2

1. Es de gran importancia utilizar un **algoritmo** durante la planeación de un programa para definir los pasos necesarios para producir el resultado final deseado.

2. Tres categorías principales de las operaciones de *lenguaje algorítmico* son: *secuencia*, *decisión* e *iteración*.
3. Tres operaciones de toma de decisiones son:
 - *si condición entonces*
 acciones.
 - *si condición entonces*
 acciones.
 si no
 acciones.
 - *Según sea expresión ordinal hacer*
 lista de valores ordinales: acciones.
 - ...
 - *si no*
 acciones.
4. Tres operaciones de iteración son:
 - *mientras expresión condicional hacer*
 acciones.
 - *Hacer*
 Acciones
 mientras expresión condicional
 - *Para instrucción de asignación hasta valor final hacer en paso de incremento*
 acciones.

RESPUESTAS EXAMEN BREVE 1-3

1. La *abstracción* permite desde el principio concentrarse en el problema, sin preocuparse en los detalles de implementación a un lenguaje de programación.
2. El *refinamiento sucesivo* comienza con un algoritmo abstracto inicial y de manera paulatina, se divide en más algoritmos relacionados que proporcionan más y más detalles.
3. Se alcanza un *nivel codificable de un algoritmo* cuando todas las enuncias dos se han reducido a las operaciones de pseudocódigo que se enumeran en la *tabla 1.1*.